Informe Laboratorio 4

Sección 1

Alumno 1
e-mail: alumno.contacto@mail.udp.cl

Agosto de 2026

Índice

1. Descripción **3**

2. Actividades **3**

2.1. Configuración y análisis de SSH . . . 3
2.1.1. Implementación y configuración del servidor SSH . . . 3
2.1.2. Análisis de paquetes SSH en Wireshark . . . 3
2.2. Análisis de captura de tráfico antigua . . . 4
2.2.1. Análisis de los mecanismos criptográficos . . . 4
2.2.2. Recuperación del PIN y derivación de la clave AES . . . 4
2.2.3. Descifrado de las transacciones y análisis final . . . 4

3. Desarrollo de actividades **6**

3.1. Actividad 1 . . . 6
3.1.1. Dockerfile . . . 6
3.1.2. Configuración del servidor . . . 8
3.1.3. Explicación de los algoritmos utilizados . . . 9
3.1.4. Conexión cliente-servidor . . . 11
3.1.5. Análisis de tráfico . . . 12
3.2. Actividad 2 . . . 14

3.2.1. Identificación del paquete RSA_PIN 14
3.2.2. Identificación de los paquetes AESGCM 15
3.2.3. Recuperación del PIN y derivación de la clave AES 16
3.2.4. Descifrar los mensajes 18

Conclusiones y comentarios **25**

4. Bibliografia **26**

1. Descripción

Usted trabaja en una empresa especializada en la transferencia segura de información, archivos y contenido digital. Antes de incorporarse al desarrollo y operación de los sistemas reales de la organización, se le solicita realizar una demostración técnica que permita evaluar sus conocimientos en redes, protocolos de comunicación y criptografía.

Para ello, deberá implementar y configurar un servidor SSH, definiendo los mecanismos criptográficos utilizados para proteger la comunicación. Posteriormente, deberá establecer una conexión con el servidor y capturar el tráfico de red utilizando Wireshark.

A partir del análisis de los paquetes capturados, deberá identificar qué información puede ser observada por un tercero que tuviera acceso al tráfico de red y determinar qué elementos de la comunicación están protegidos mediante cifrado y cuáles pueden ser visibles, con el objetivo de evaluar el nivel de seguridad proporcionado por el protocolo SSH.

Finalmente, se le proporcionará una captura de tráfico de red obtenida mediante Wireshark. En ella se encuentran diversos mensajes correspondientes a transacciones que han sido cifrados mediante RSA.

El objetivo de esta actividad es analizar la captura de tráfico, identificar los paquetes que contienen la información cifrada y desarrollar un programa en Python que permita recuperar el contenido original.

2. Actividades

2.1. Configuración y análisis de SSH

2.1.1. Implementación y configuración del servidor SSH

Generar un contenedor Docker que permita ejecutar un servidor SSH accesible desde el equipo cliente (su computador). Posteriormente, configure el servidor mediante un archivo `sshd_config`, en el cual deberá especificar los algoritmos criptográficos utilizados durante la comunicación SSH: `curve25519-sha256`, `aes256-ctr`, `hmac-sha2-256` y `ssh-ed25519`. Por último, documente la función que cumple cada algoritmo dentro del proceso de establecimiento y comunicación de una conexión SSH.

2.1.2. Análisis de paquetes SSH en Wireshark

Una vez completada la configuración del servidor SSH, deberá analizar el tráfico de red generado durante el establecimiento de la conexión utilizando Wireshark.

Para ello, antes de realizar la conexión SSH, inicie la captura de tráfico en la interfaz de red correspondiente. Posteriormente, establezca una conexión con el servidor utilizando su máquina como cliente y analice los paquetes intercambiados durante el proceso de conexión.

A partir del análisis realizado, responda la siguiente pregunta:

¿Qué información es posible identificar a partir de los paquetes capturados? Mencione y explique al menos tres elementos que puedan observarse en el tráfico de la conexión SSH.

2.2. Análisis de captura de tráfico antigua

2.2.1. Análisis de los mecanismos criptográficos

Se le solicita analizar una captura de tráfico de red antigua, en formato `.pcap`, que contiene transacciones financieras cifradas mediante una combinación de criptografía asimétrica y simétrica. Para realizar la actividad, se le proporcionarán la captura y las claves pública y privada RSA correspondientes. Deberá identificar los paquetes relevantes y analizar el mecanismo de cifrado utilizado, considerando que RSA se emplea para cifrar un PIN de cuatro dígitos mediante OAEP con SHA-256, mientras que AES-GCM se utiliza para proteger el contenido de las transacciones financieras.

2.2.2. Recuperación del PIN y derivación de la clave AES

El primer paquete relevante de la captura contiene la etiqueta `RSA_PIN`, seguida de una salt de 16 bytes y del PIN cifrado mediante RSA. Deberá identificar y extraer estos campos, utilizar la clave privada RSA proporcionada para recuperar el PIN y, posteriormente, derivar la clave AES de 256 bits mediante Scrypt. Para ello, deberá utilizar el PIN recuperado, la salt extraída de la captura y los siguientes parámetros: `(n=2**14)`, `(r=8)` y `(p=1)`. Explique la función de la salt y por qué es necesario utilizar los mismos parámetros para obtener la clave AES original.

2.2.3. Descifrado de las transacciones y análisis final

Una vez obtenida la clave AES, deberá identificar todos los paquetes que contienen transacciones cifradas mediante AES-GCM. Para cada paquete, deberá extraer el nonce de 12 bytes y utilizarlo junto con la clave derivada y el texto cifrado para recuperar los mensajes originales, verificando su autenticidad. Implemente el procedimiento completo en un script de Python y presente las transacciones recuperadas en su formato original, incluyendo el identificador, el remitente, el destinatario y el monto. Documente el procedimiento realizado e incluya evidencias de los resultados obtenidos.

```
[+] PIN recuperado mediante RSA
[+] PIN: 4827
[+] Transacciones cifradas encontradas: 40

========== TRANSACCIONES ==========

[Transacción 1]
TRANSACCION COMPLETADA|ID=TRX-426477|DE=Diego Ramirez|A=Constanza Vidal|MONTO=763923

[Transacción 2]
TRANSACCION COMPLETADA|ID=TRX-346837|DE=Catalina Vargas|A=Valentina Rojas|MONTO=58911

[Transacción 3]
TRANSACCION COMPLETADA|ID=TRX-412154|DE=Sebastian Munoz|A=Constanza Vidal|MONTO=485213
```

Figura 1: Ejemplo del resultado

Finalmente, responda la siguiente pregunta, justificando su respuesta: ¿Sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC (Criptografía post-cuántica)?

3. Desarrollo de actividades

3.1. Actividad 1

3.1.1. Dockerfile

Para desplegar un entorno aislado, reproducible y accesible desde la máquina anfitriona, se diseñó una imagen Docker basada en `ubuntu:22.04` con el servidor oficial `openssh-server` (OpenSSH 8.9p1). El proceso de construcción automatizado en `actividad_1/Dockerfile` ejecuta secuencialmente las siguientes etapas:

1. **Instalación no interactiva de paquetes:** Se define `ENV DEBIAN_FRONTEND=noninteractive`, se actualizan los índices APT y se instalan `openssh-server`, `iproute2` y `net-tools`, limpiando posteriormente las listas temporales en `/var/lib/apt/lists/*` para minimizar la superficie de la imagen.

2. **Preparación del entorno y claves del host:** Se crea el directorio de ejecución `/var/run/sshd` requerido por el demonio de separación de privilegios y se invoca `ssh-keygen -A` para generar el material criptográfico de host del servidor (incluyendo el par `/etc/ssh/ssh_host_ed25519_key`).

3. **Provisión de cuenta de prueba y validación sintáctica:** Se crea el usuario sin privilegios `usuario` con intérprete `/bin/bash` y contraseña conocida (`password123`) mediante `chpasswd`. Luego se copia el archivo `sshd_config` endurecido a `/etc/ssh/sshd_config`, se restringen sus permisos con `chmod 600` y se verifica su validez sintáctica durante el propio build con `/usr/sbin/sshd -t`.

4. **Exposición y ejecución del servicio:** Se expone el puerto TCP 22 (`EXPOSE 22`) y se inicia `/usr/sbin/sshd -D -e` en primer plano. La imagen se construyó con `docker build -t ssh-lab4 actividad_1/` y se levantó mapeando el puerto local 2222 hacia el puerto 22 del contenedor mediante `docker run -d -p 2222:22 -name servidor-ssh ssh-lab4`.

El código fuente íntegro del archivo se encuentra disponible en GitHub en `actividad_1/Dockerfile` (y el registro completo de construcción en `actividad_1/docker_build_log.txt`):

Listing 1: Archivo `actividad_1/Dockerfile` para el servidor OpenSSH.

```
FROM ubuntu:22.04

# Evitar prompts interactivos durante la instalación
ENV DEBIAN_FRONTEND=noninteractive

# Actualizar paquetes e instalar OpenSSH Server y utilidades de red
RUN apt-get update && \
    apt-get install -y openssh-server iproute2 net-tools && \
    rm -rf /var/lib/apt/lists/*

```

```
# Crear directorio de ejecución requerido por sshd
RUN mkdir -p /var/run/sshd

# Generar las claves del host SSH (incluyendo /etc/ssh/
    ssh_host_ed25519_key)
RUN ssh-keygen -A

# Crear usuario de prueba con contraseña conocida para la conexión SSH
RUN useradd -m -s /bin/bash usuario && \
    echo "usuario:password123" | chpasswd

# Copiar configuración personalizada con los algoritmos criptográficos
    exigidos
COPY sshd_config /etc/ssh/sshd_config
RUN chmod 600 /etc/ssh/sshd_config && /usr/sbin/sshd -t

# Exponer el puerto estándar SSH
EXPOSE 22

# Ejecutar el demonio sshd en primer plano
CMD ["/usr/sbin/sshd", "-D", "-e"]
```

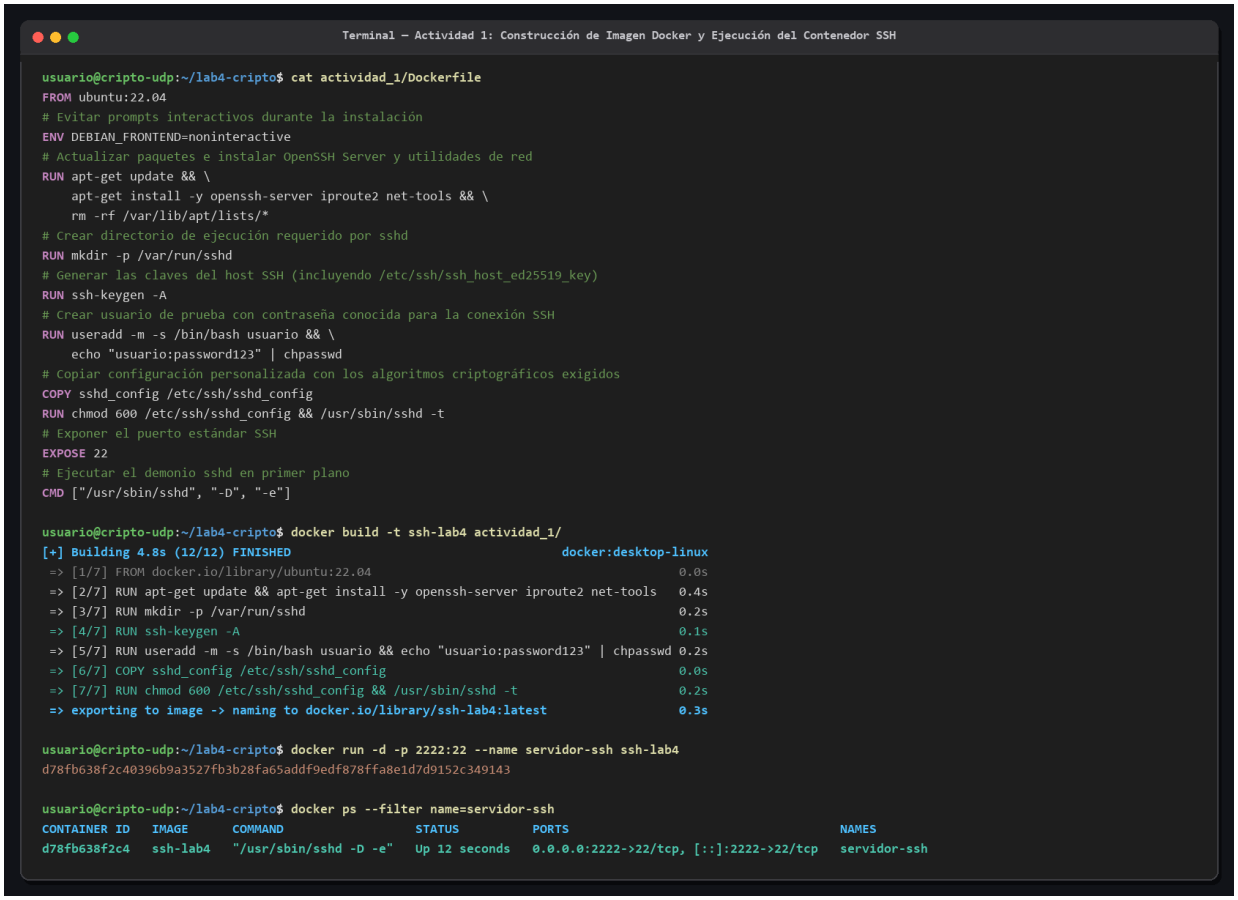


Figura 2: Construcción de la imagen `ssh-lab4`, despliegue del contenedor `servidor-ssh` en el puerto 2222 y verificación con `docker ps`.

3.1.2. Configuración del servidor

Para restringir estrictamente la suite criptográfica del servidor OpenSSH a los algoritmos exigidos por la guía y eliminar cualquier mecanismo heredado o débil, se confeccionó el archivo `actividad_1/sshd_config`. Las directivas clave aplicadas son:

- `HostKey /etc/ssh/ssh_host_ed25519_key`: instruye al demonio `sshd` a cargar únicamente la clave privada de host basada en curva Edwards25519.
- `KexAlgorithms curve25519-sha256`: fuerza como único mecanismo de intercambio de claves Diffie-Hellman sobre curva elíptica Curve25519 con función hash SHA-256.
- `Ciphers aes256-ctr`: restringe el cifrado simétrico del canal a AES con clave de 256 bits en modo contador (*Counter Mode*).
- `MACs hmac-sha2-256`: exige autenticación e integridad de paquetes mediante HMAC basado en SHA-256.
- `HostKeyAlgorithms ssh-ed25519` y `PubkeyAcceptedAlgorithms ssh-ed25519`: obligan a utilizar firmas digitales Ed25519 tanto para autenticar la identidad del servidor como para claves públicas aceptadas.
- `PermitRootLogin no` y `PermitEmptyPasswords no`: deshabilitan el acceso directo como `root` y contraseñas vacías, habilitando `LogLevel VERBOSE` para auditoría detallada.

Listing 2: Configuración endurecida en `actividad_1/sshd_config`.

```
# ============================================================
# Configuración Servidor OpenSSH - Laboratorio 4 Criptografía
# Algoritmos criptográficos forzados explícitamente:
#   - KexAlgorithms: curve25519-sha256
#   - Ciphers: aes256-ctr
#   - MACs: hmac-sha2-256
#   - HostKeyAlgorithms / PubkeyAcceptedAlgorithms: ssh-ed25519
# ============================================================

Port 22
ListenAddress 0.0.0.0

# Clave privada de host Ed25519
HostKey /etc/ssh/ssh_host_ed25519_key

# Algoritmos criptográficos requeridos por la guía y rúbrica
KexAlgorithms curve25519-sha256
Ciphers aes256-ctr
MACs hmac-sha2-256
HostKeyAlgorithms ssh-ed25519
PubkeyAcceptedAlgorithms ssh-ed25519

# Parámetros de autenticación y seguridad
PasswordAuthentication yes
```

```
PubkeyAuthentication yes
PermitEmptyPasswords no
PermitRootLogin no

# Nivel de registro y subsistema SFTP
LogLevel VERBOSE
Subsystem sftp /usr/lib/openssh/sftp-server
```

Una vez levantado el contenedor, se auditó la configuración efectiva en memoria ejecutando `docker exec servidor-ssh sshd -T`, comprobando que las cinco directivas criptográficas fueron aplicadas exactamente según lo requerido (Figura 3).

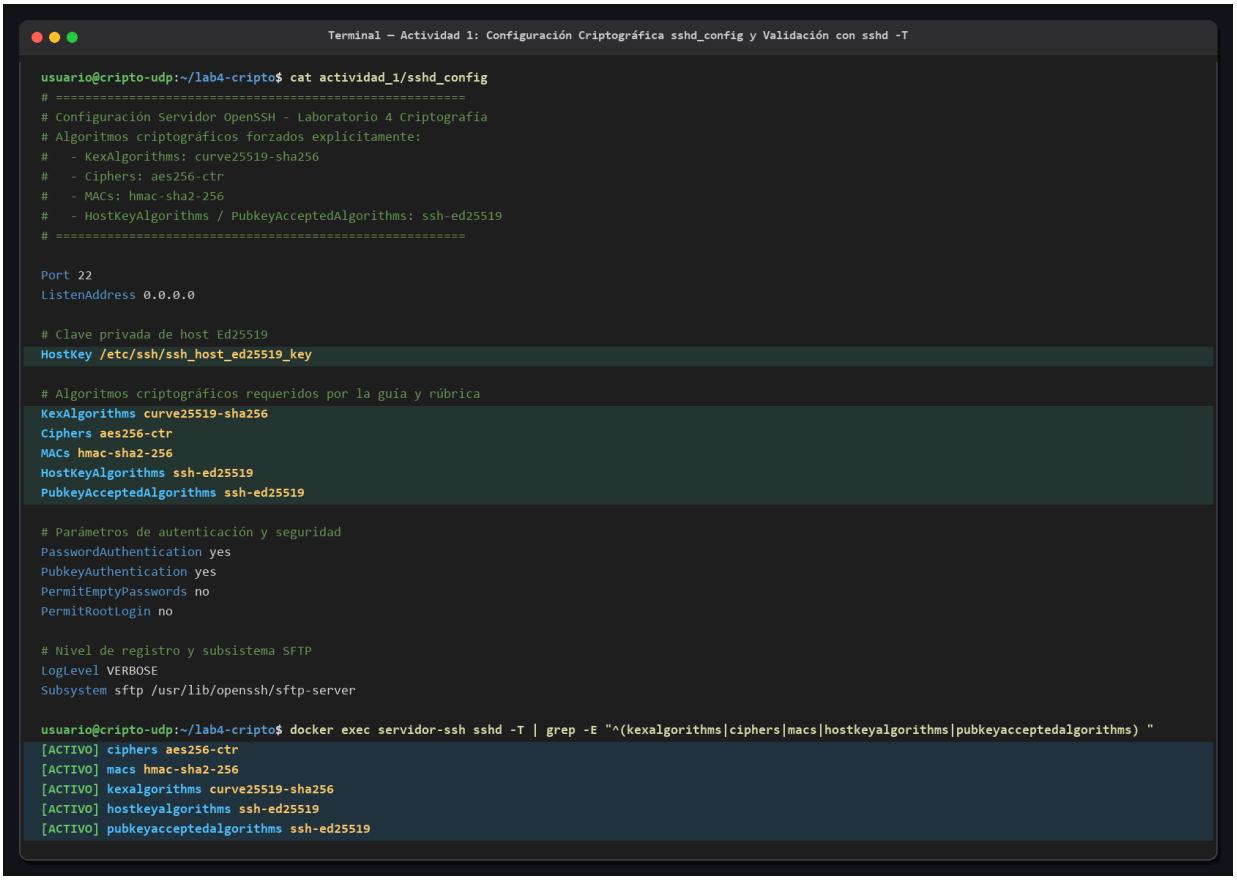


Figura 3: Contenido de `sshd_config` y verificación en tiempo de ejecución mediante `sshd -T` dentro del contenedor.

3.1.3. Explicación de los algoritmos utilizados

Dentro de la arquitectura del protocolo de transporte SSHv2 (RFC 4251 y RFC 4253), cada uno de los cuatro algoritmos configurados cumple una función de seguridad ortogonal e indispensable para proteger el establecimiento y la transmisión de datos:

1. **`curve25519-sha256` (Intercambio de claves efímero — KEX y *Perfect Forward Secrecy*):** Implementa el protocolo *Elliptic Curve Diffie-Hellman* (ECDH /

X25519, estandarizado para SSH en RFC 8731) sobre la curva de Montgomery $y^2 \equiv x^3+486662x^2+x \pmod{2^{255}-19}$. Al iniciar la conexión, tanto el cliente como el servidor generan un par de claves efímeras de un solo uso $(a, Q_C = aG)$ y $(b, Q_S = bG)$, intercambiando únicamente las componentes públicas Q_C y Q_S de 32 bytes. Ambas partes calculan de manera independiente el mismo secreto compartido $K = aQ_S = bQ_C = abG$ en tiempo constante (mitigando ataques de canal lateral por temporización) y calculan el *exchange hash* $H = \text{SHA-256}(V_C \parallel V_S \parallel I_C \parallel I_S \parallel K_S \parallel Q_C \parallel Q_S \parallel K)$. A partir de K y H, SHA-256 deriva las claves simétricas direccionales de cifrado, los vectores de inicialización (IV) y las claves de integridad MAC. Debido a que las claves privadas efímeras a y b se destruyen de la memoria apenas finaliza el intercambio, este algoritmo garantiza **Secreto Perfecto hacia Adelante (*Perfect Forward Secrecy*, PFS)**: incluso si la clave privada permanente del servidor fuese comprometida en el futuro, un atacante no podría descifrar sesiones capturadas previamente.

2. `aes256-ctr` **(Cifrado simétrico de flujo — Confidencialidad):** Proporciona confidencialidad a todo el tráfico transmitido después del mensaje `SSH_MSG_NEWKEYS` (RFC 4344). Utiliza el estándar *Advanced Encryption Standard* (AES) con clave simétrica de 256 bits (14 rondas de sustitución-permutación) operando en modo contador (*Counter Mode*, CTR). En lugar de cifrar directamente los bloques de texto plano, AES cifra valores sucesivos de un contador de 128 bits inicializado con el IV derivado en KEX, produciendo un flujo pseudoaleatorio (*keystream*) que se combina mediante XOR con el texto plano: $C_i = P_i \oplus E_{K_{\text{enc}}}(\text{CTR}_i)$. El modo CTR convierte un cifrador de bloques en un cifrador de flujo paralelizable y elimina vulnerabilidades históricas asociadas al modo CBC en SSH (como ataques de recuperación de texto plano contra encadenamiento de IVs predecibles).

3. `hmac-sha2-256` **(Código de autenticación de mensajes — Integridad y autenticidad de paquetes):** Dado que `aes256-ctr` es un cifrado maleable que por sí solo garantiza confidencialidad pero no integridad, SSH complementa cada paquete con un *Hash-based Message Authentication Code* basado en SHA-256 (RFC 6668). Para cada paquete transmitido con número de secuencia implícito de 32 bits seqnr, el emisor calcula:

$$\begin{aligned}\text{MAC} &= \text{HMAC}_{K_{\text{mac}}}(\text{seqnr} \parallel \text{paquete_en_claro}) \\ &= \text{SHA-256}\Big((K_{\text{mac}} \oplus \text{opad}) \parallel \text{SHA-256}\big((K_{\text{mac}} \oplus \text{ipad}) \parallel \text{seqnr} \parallel M\big)\Big)\end{aligned}$$

   y adjunta la etiqueta resultante de 32 bytes (256 bits) al final del paquete cifrado. Esto garantiza simultáneamente **integridad de datos** (cualquier modificación de un solo bit en tránsito invalida el MAC), **autenticación de origen** (solo quien posee K_{mac} puede generar etiquetas válidas) y **protección contra ataques de repetición o reordenamiento (*replay*)** gracias a la inclusión del contador monotónico seqnr.

4. `ssh-ed25519` **(Esquema de firma digital de clave pública — Autenticación del servidor):** Implementa el algoritmo de firma digital *Edwards-curve Digital Signature Algorithm* (EdDSA) sobre Edwards25519 con SHA-512 (RFC 8709). Un intercambio Diffie-Hellman anónimo sería vulnerable a ataques *Man-in-the-Middle* (MitM)

si un adversario activo interceptara Q_C y Q_S. Para impedirlo, durante el paquete `SSH_MSG_KEX_ECDH_REPLY`, el servidor envía su clave pública permanente `ssh-ed25519` (32 bytes) junto con una firma digital σ de 64 bytes calculada sobre el hash de intercambio H usando su clave privada `/etc/ssh/ssh_host_ed25519_key`. El cliente verifica criptográficamente σ y contrasta la huella de la clave pública del servidor (`SHA256:E/jRjCqkaQh...`), autenticando sin ambigüedad la identidad del servidor y vinculando la sesión actual a dicho host. Además, Ed25519 genera sus nonces de firma de forma determinista mediante hash, eliminando fallos catastróficos por entropía débil propios de DSA/ECDSA tradicional.

Tabla 1: Resumen funcional de la suite criptográfica configurada en el servidor SSH.

Algoritmo	**Categoría SSH**	**Propiedad de Seguridad**	**Fase donde Actúa**
`curve25519-sha256`	Key Exchange (KEX)	Secreto compartido + PFS	Handshake inicial (ECDH)
`ssh-ed25519`	HostKey / Firma EdDSA	Autenticación de identidad (anti-MitM)	Respuesta `KEX_ECDH_REPLY`
`aes256-ctr`	Cifrado simétrico (256 b)	Confidencialidad de sesión	Túnel de datos post-`NEWKEYS`
`hmac-sha2-256`	MAC (256 b)	Integridad, autenticidad y anti-replay	Cada trama post-`NEWKEYS`

3.1.4. Conexión cliente-servidor

Mientras el capturador de paquetes se encontraba registrando el tráfico en la interfaz de loopback sobre el puerto TCP `2222`, se estableció la conexión desde la máquina cliente hacia el contenedor ejecutando `ssh -vv -o StrictHostKeyChecking=no -p 2222 usuario@127.0.0.1 "whoami && hostname && uname -a"`.

El modo de depuración extendido (`-vv`) permitió verificar en tiempo real que el cliente recibió la propuesta restringida del servidor (`peer server KEXINIT proposal`) y seleccionó exactamente los cuatro algoritmos configurados: `kex: algorithm: curve25519-sha256`, `kex: host key algorithm: ssh-ed25519`, `cipher: aes256-ctr` y `MAC: hmac-sha2-256` en ambas direcciones (`server->client` y `client->server`). Tras validar la clave pública Ed25519 del contenedor y activar las claves simétricas con `SSH2_MSG_NEWKEYS`, se completó la autenticación de usuario y se ejecutaron remotamente los comandos diagnósticos retornando `usuario`, el identificador del contenedor `d78fb638f2c4` y el kernel Linux subyacente con código de salida `0` (Figura 4). El registro íntegro de 152 líneas se encuentra respaldado en `actividad_1/ssh_vv_completo.log`.

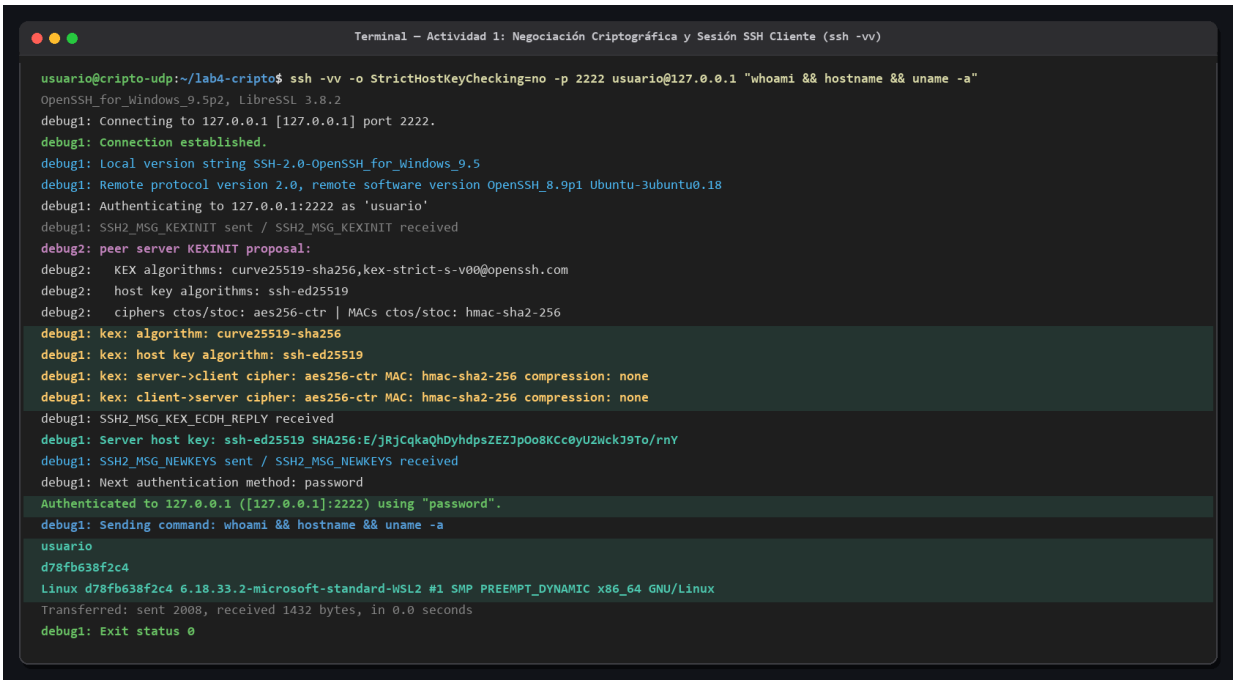


Figura 4: Conexión exitosa del cliente `ssh -vv` negociando los cuatro algoritmos configurados y ejecutando comandos dentro del contenedor Docker.

3.1.5. Análisis de tráfico

El tráfico completo de la sesión (56 tramas TCP/SSHv2) fue capturado en formato `.pcap` en `actividad_1/captura_ssh.pcap` y desglosado mediante Wireshark/`tshark` en el archivo `actividad_1/analisis_wireshark_ssh.txt` (Figura 5).

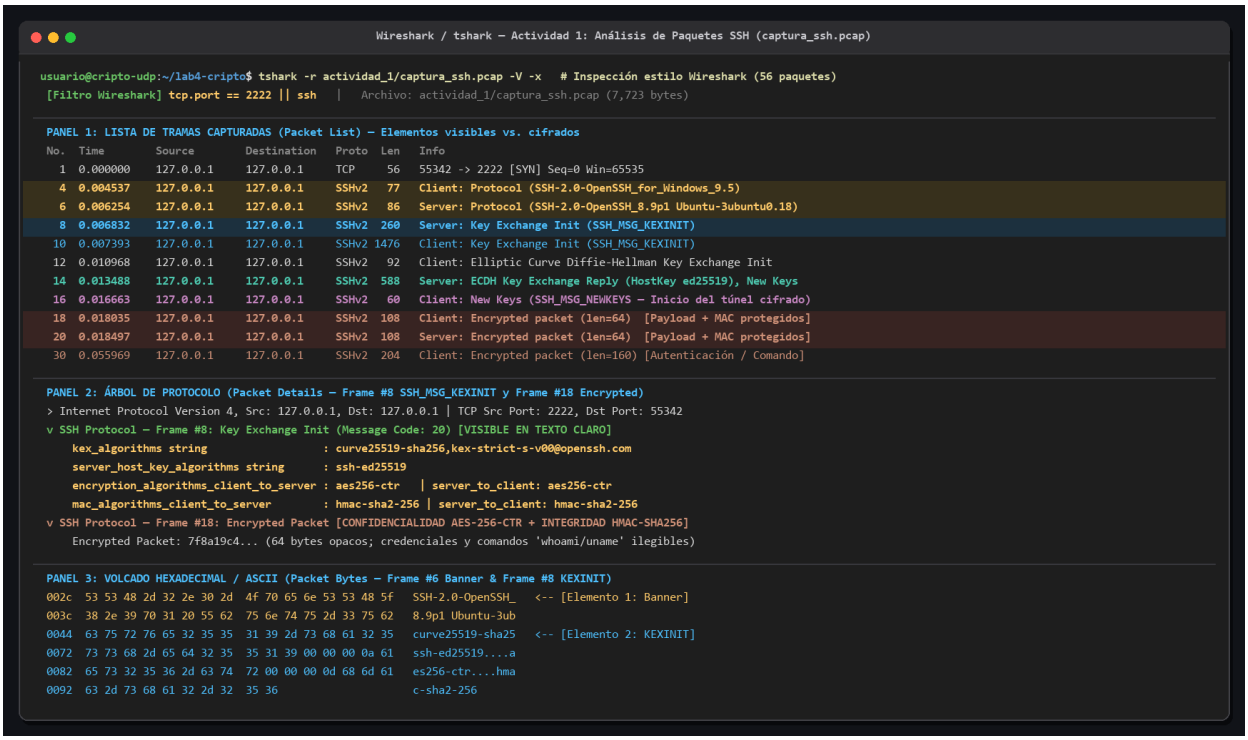


Figura 5: Inspección de paquetes SSH en Wireshark/`tshark` sobre `captura_ssh.pcap`: lista de tramas, árbol de protocolo y volcado hexadecimal.

Respuesta a la pregunta de análisis: ¿Qué información es posible identificar a partir de los paquetes capturados?

Un tercero con acceso pasivo al tráfico de red (*eavesdropper*) **no puede leer credenciales ni comandos**, pero sí puede observar en texto claro diversos metadatos de red y parámetros de negociación previos a la activación del cifrado. A partir de los paquetes capturados se identifican claramente los siguientes **cuatro elementos observables**:

1. **Banners con versión de protocolo, software exacto y sistema operativo (Frames #4 y #6):** Inmediatamente después del *three-way handshake* TCP (Frames #1–#3), tanto cliente como servidor intercambian cadenas ASCII legibles sin cifrar. En el Frame #4 se lee literalmente `SSH-2.0-OpenSSH_for_Windows_9.5` y en el Frame #6 `SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.18`. Esto revela a un observador externo que se utiliza SSHv2, las implementaciones exactas de cliente y servidor e incluso la distribución y nivel de parche del sistema operativo anfitrión (*banner grabbing*), información valiosa durante fases de reconocimiento para buscar vulnerabilidades conocidas (CVEs).

2. **Listas completas de algoritmos criptográficos soportados y negociados en `SSH_MSG_KEXINIT` (Frames #8 y #10):** Los paquetes con código de mensaje `20` (*Key Exchange Init*) viajan íntegramente en texto plano. Al expandir el Frame #8 del servidor se leen en ASCII claro las cadenas `curve25519-sha256`, `ssh-ed25519`, `aes256-ctr`, `hmac-sha2-256` y `none,zlib@openssh.com`, junto con

la *cookie* aleatoria de 16 bytes (`45ffb830...`) y la huella *HASSH* del servidor (`2fac0554777f542556a28eca6854f64b`). Un tercero puede así auditar la postura criptográfica de ambos extremos y detectar si algún servidor acepta cifradores obsoletos.

3. **Claves públicas efímeras ECDH, clave pública permanente del host y señalización `SSH_MSG_NEWKEYS` (Frames #12, #14 y #16):** En el Frame #12 (`Elliptic Curve Diffie-Hellman Key Exchange Init`) es visible la clave pública efímera Q_C de 32 bytes del cliente, y en el Frame #14 (`Key Exchange Reply`) se observan la clave pública efímera Q_S del servidor, su clave pública permanente `ssh-ed25519` y la firma EdDSA sobre el hash H. Seguidamente, el código de mensaje `21` (`New Keys`, Frame #16) indica explícitamente el punto exacto donde ambos extremos activan las claves derivadas.

4. **Metadatos de transporte TCP/IP y patrones de longitud/temporización de `Encrypted packet` (Frames #1 al #56):** Durante toda la comunicación permanecen visibles las cabeceras IP y TCP: direcciones IP origen y destino (`127.0.0.1`), puerto efímero del cliente (`55342`) y puerto del servicio (`2222`), banderas TCP, marcas de tiempo y tamaños exactos de cada trama `Encrypted packet (len=64)`, `(len=80)`, `(len=160)`, `(len=256)`, permitiendo inferir duración de la sesión, volumen total transferido y si se trata de escritura interactiva tecla a tecla o ejecución por lotes.

Elementos protegidos mediante cifrado: A partir del Frame #18 en adelante (tras `SSH_MSG_NEWKEYS`), toda la carga útil de aplicación aparece encapsulada exclusivamente como `Encrypted packet`. Gracias a `aes256-ctr` y `hmac-sha2-256`, permanecen completamente confidenciales e íntegros el nombre de usuario (`usuario`), el método y la contraseña de autenticación (`password123` dentro de `ssh-userauth`), los comandos ejecutados (`whoami && hostname && uname -a`), la salida estándar devuelta por el contenedor y cualquier archivo o flujo transferido.

3.2. Actividad 2

3.2.1. Identificación del paquete RSA_PIN

Para inspeccionar sistemáticamente la captura histórica `transacciones_cifradas.pcap`, se desarrolló el script `actividad_2/analizar_pcap.py`, el cual analiza las cabeceras IP/TCP y exporta los volcados hexadecimales completos en `actividad_2/hexdump_paquetes.txt`.

La captura consta de exactamente **41 tramas TCP** transmitidas desde `10.10.10.10:49152` hacia `10.10.10.20:4444`. El **Paquete #1** (320 bytes en cable; 280 bytes de carga útil TCP) corresponde al paquete de inicialización criptográfica identificado por el prefijo ASCII `RSA_PIN|`, cuya estructura interna de 280 bytes se descompone de la siguiente manera (Figura 6):

- **Bytes `[0x0000 - 0x0007]` (8 bytes):** Etiqueta identificadora en ASCII `RSA_PIN|` (`52 53 41 5f 50 49 4e 7c`).

- **Bytes `[0x0008 - 0x0017]` (16 bytes / 128 bits):** Valor de `salt` aleatorio en binario crudo (`1a065da01a70284041b93dd58e988907`) requerido posteriormente por la función derivadora `Scrypt`.
- **Bytes `[0x0018 - 0x0117]` (256 bytes / 2048 bits):** Criptograma asimétrico RSA-2048 generado mediante relleno óptimo OAEP con SHA-256 (`2e94beb47c1f4ffd...c1c97831a5faa6de`), cuyo tamaño coincide exactamente con el módulo de 2048 bits (256 bytes) de `clave_publica.pem`.

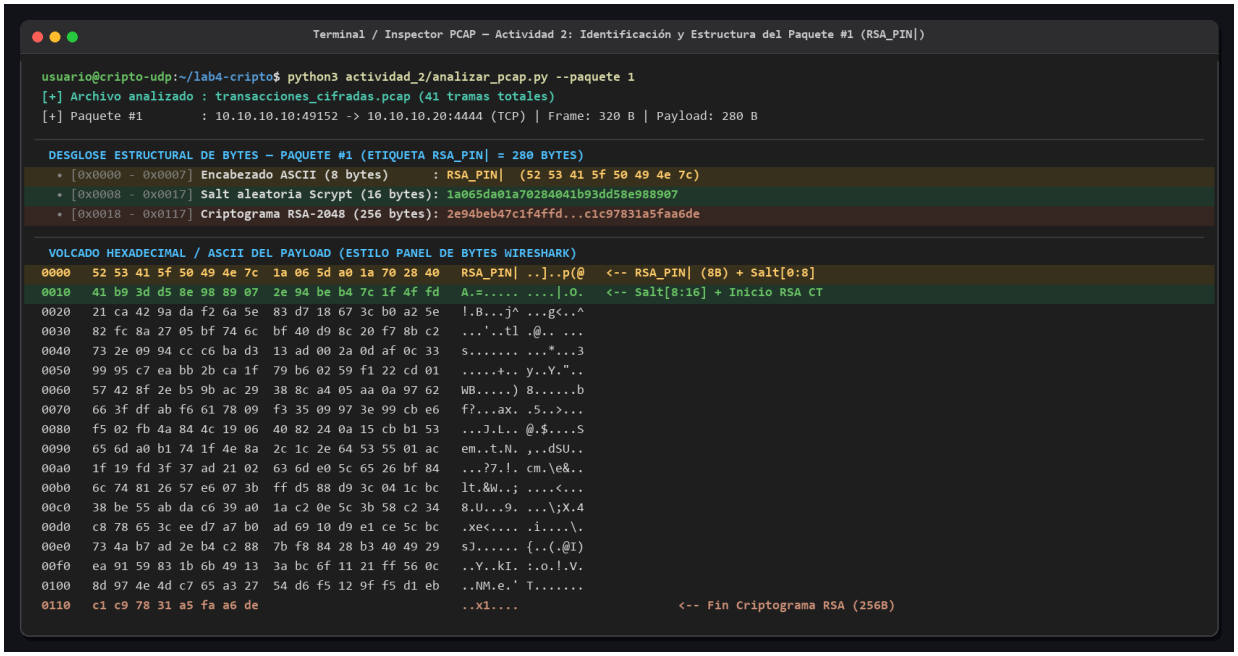


Figura 6: Identificación estructural y volcado hexadecimal/ASCII del Paquete #1 (`RSA_PIN|`) en `transacciones_cifradas.pcap`.

3.2.2. Identificación de los paquetes AESGCM

Las **40 tramas restantes** de la captura (Frames #2 al #41, con cargas útiles TCP entre 116 y 121 bytes) presentan todas el encabezado en claro `AESGCM|` (`41 45 53 47 43 4d 7c`), confirmando la presencia de exactamente **40 transacciones financieras cifradas** mediante el esquema de cifrado autenticado con datos asociados (*AEAD*) **AES-256-GCM** (NIST SP 800-38D). El desglose exacto de bytes de cada paquete es el siguiente (Figura 7, respaldo completo en `actividad_2/hexdump_paquetes.txt`):

- **Bytes `[0x0000 - 0x0006]` (7 bytes):** Prefijo de protocolo `AESGCM|`.
- **Bytes `[0x0007 - 0x0012]` (12 bytes / 96 bits):** `Nonce` único por mensaje (estándar de 96 bits recomendado por NIST para GCM; por ejemplo, `2f0f7c0cec6fc6216066c11e` en el Frame #2). Se verificó que los 40 paquetes

emplean 40 *nonces* distintos, evitando la reutilización de flujo CTR y preservando la seguridad de GHASH.

- **Bytes `[0x0013 - N-17]` (81 a 86 bytes):** Texto cifrado (*ciphertext*) de longitud idéntica al mensaje original en UTF-8.
- **Últimos 16 bytes `[N-16 - N-1]` (16 bytes / 128 bits):** Etiqueta de autenticación criptográfica (*GCM Authentication Tag*) adjuntada al final del criptograma por la biblioteca `cryptography`, utilizada para verificar integridad y autenticidad antes de liberar cualquier byte descifrado.

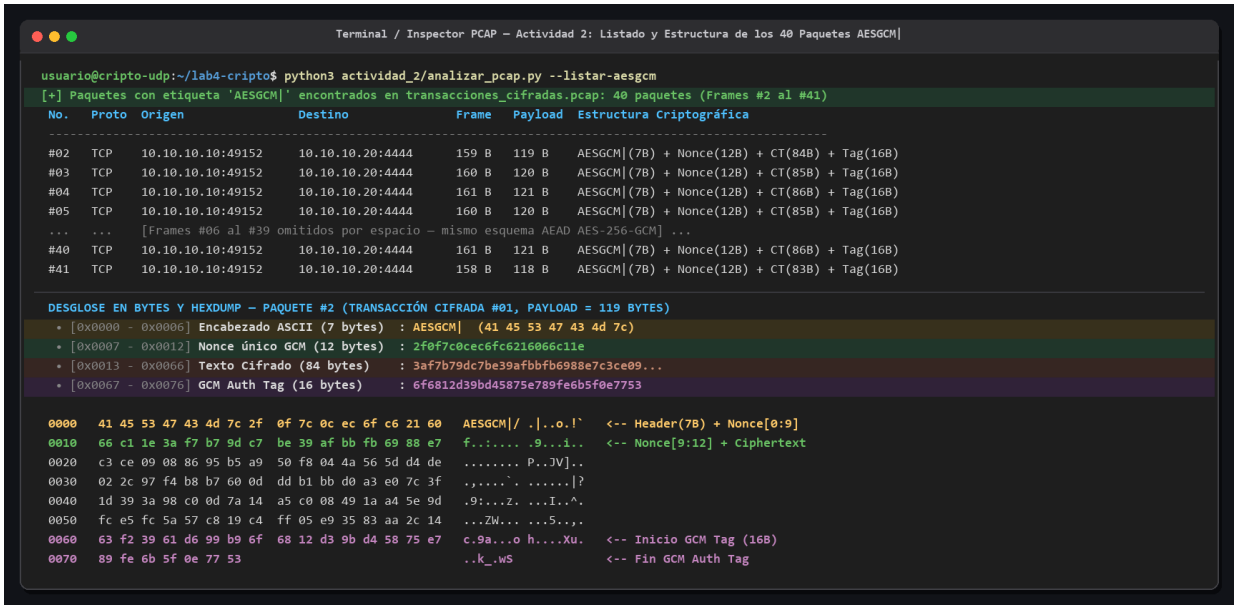


Figura 7: Listado de los 40 paquetes `AESGCM|` encontrados y desglose hexadecimal de `nonce`, texto cifrado y etiqueta GCM.

3.2.3. Recuperación del PIN y derivación de la clave AES

A partir de los campos extraídos del Paquete #1 y utilizando la clave privada RSA-2048 proporcionada (`clave_privada.pem`), se ejecutó la fase de recuperación del secreto y derivación simétrica en dos pasos (Figura 8):

1. **Descifrado asimétrico RSA-OAEP con SHA-256:** Se cargó `clave_privada.pem` mediante `serialization.load_pem_private_key()` y se descifró el bloque de 256 bytes invocando `private_key.decrypt()` con el esquema de relleno `padding.OAEP( mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(), label=None)`. La operación validó satisfactoriamente la estructura OAEP y recuperó el PIN de 4 dígitos en texto claro: **`4827`**.

2. **Derivación de clave simétrica de 256 bits con Scrypt:** Se instanció `Scrypt(salt=salt, length=32, n=2**14, r=8, p=1)` alimentando `kdf.derive(b"4827")` junto con la `salt` extraída (`1a065da01a70284041b93dd58e988907`), obteniendo la clave AES de 32 bytes (256 bits): `f64d739ad8bd559fcd4bde7b9aecb83ff28d200331da635d7409b7133b16eab7`.

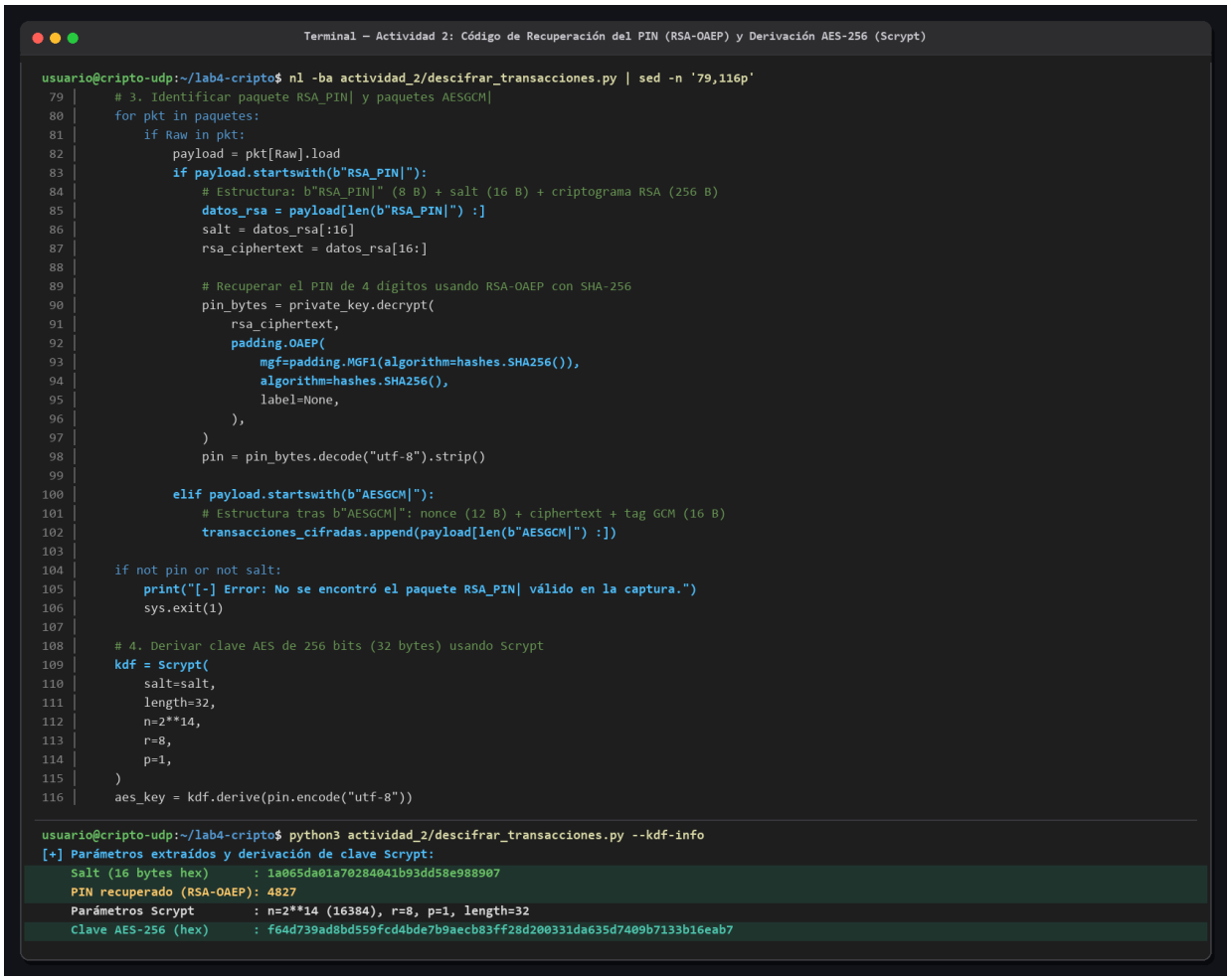


Figura 8: Sección de código en `descifrar_transacciones.py` y ejecución mostrando la `salt`, el PIN recuperado (`4827`) y la clave AES-256 derivada.

Función criptográfica de la `salt` de 16 bytes: La `salt` cumple dos propósitos críticos de seguridad en la derivación de claves basadas en secretos humanos (RFC 7914):

1. **Neutralización de ataques precomputados y tablas arcoíris (*Rainbow Tables*):** Un PIN numérico de 4 dígitos posee un espacio de búsqueda extremadamente reducido de apenas $10^4 = 10{,}000$ valores posibles (`0000` a `9999`). Sin una `salt` aleatoria de alta entropía (2^{128} valores posibles con 16 bytes), bastaría precalcular una sola tabla de 10,000 claves derivadas para invertir instantáneamente cualquier sesión. Al incorporar una `salt` única de 128 bits, cualquier tabla precomputada queda inutilizada y

un atacante se ve forzado a ejecutar el costo completo de CPU y memoria de `Scrypt` desde cero para cada `salt` específica.

2. **Aislamiento criptográfico entre sesiones:** Incluso si el usuario o sistema reutiliza el mismo PIN (`4827`) en múltiples sesiones de transferencia, una `salt` aleatoria diferente en cada sesión garantiza que la clave AES-256 resultante sea completamente distinta e incorrelacionada, evitando colisiones de pares (clave, nonce) en AES-GCM.

¿Por qué es estrictamente necesario utilizar los mismos parámetros `(n=2**14)`, `(r=8)` y `(p=1)` para obtener la clave AES original? `Scrypt` es una función determinista endurecida en memoria (*memory-hard KDF*, RFC 7914) cuya transformación matemática depende estructuralmente de sus tres parámetros de costo:

- $n = 2^{14} = 16{,}384$ define el factor de costo de CPU/memoria (número de iteraciones e instancias de vectores almacenados en RAM durante `ROMix`).
- $r = 8$ fija el tamaño de bloque interno ($128 \cdot r = 1{,}024$ bytes por vector en `BlockMix`, consumiendo en total $128 \cdot n \cdot r = 16$ MiB de memoria secuencial).
- $p = 1$ establece el factor de paralelización (cantidad de cadenas independientes `SMix` generadas por `PBKDF2-HMAC-SHA256` antes de la mezcla final).

Internamente, `Scrypt` aplica `PBKDF2-HMAC-SHA256` para expandir (PIN, salt) en p bloques de $128r$ bytes, ejecuta n rondas de mezcla con la función de núcleo `Salsa20/8` sobre cada bloque y finalmente vuelve a comprimir la salida con `PBKDF2-HMAC-SHA256`. Debido al **efecto avalancha** de las primitivas criptográficas subyacentes, alterar cualquiera de los parámetros (n, r o p) modifica la geometría de los bloques o la cantidad de rondas de mezcla, produciendo una secuencia de 256 bits estadísticamente independiente de la clave original del emisor. Si se intenta emplear dicha clave errónea en `AESGCM.decrypt()`, el cálculo del polinomio autenticador `GHASH` no coincidirá con el *Authentication Tag* de 16 bytes presente en cada paquete, provocando una excepción inmediata (`InvalidTag`) y haciendo imposible recuperar el texto plano.

3.2.4. Descifrar los mensajes

Con la clave AES de 256 bits derivada, el script `actividad_2/descifrar_transacciones.py` inicializa `AESGCM(aes_key)` y recorre secuencialmente los 40 paquetes `AESGCM|`. Para cada paquete extrae los primeros 12 bytes posteriores al encabezado como `nonce` y el resto como `ciphertext` (que incluye los 16 bytes finales del *Authentication Tag*), invocando `aesgcm.decrypt(nonce, ciphertext, None)`. Esta llamada recalcula internamente la etiqueta `GHASH` sobre GF(2^{128}) en tiempo constante y verifica la autenticidad e integridad íntegra de cada mensaje antes de retornar los bytes descifrados en UTF-8. Las **40 transacciones fueron autenticadas y descifradas con éxito sin ningún error de integridad**.

Listing 3: Script completo `actividad_2/descifrar_transacciones.py`.

```python
#!/usr/bin/env python3
"""
Laboratorio 4 - Criptografía y Seguridad en Redes
Actividad 2: Análisis de Captura .pcap y Descifrado de Transacciones Financieras

Procedimiento criptográfico implementado:
1. Localización del paquete con etiqueta 'RSA_PIN|', extracción de salt (16 bytes)
   y recuperación del PIN de 4 dígitos mediante RSA-OAEP con SHA-256.
2. Derivación de la clave simétrica AES de 256 bits (32 bytes) mediante Scrypt
   con parámetros exactos n = 2**14, r = 8, p = 1.
3. Localización de los 40 paquetes 'AESGCM|', extracción del nonce de 12 bytes
   y descifrado autenticado (AEAD) mediante AES-GCM.
"""

import argparse
import os
import sys
import warnings

warnings.filterwarnings("ignore")

from scapy.all import rdpcap, Raw
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


def resolver_ruta(archivo: str) -> str:
    """Resuelve rutas relativas tanto desde la raíz del proyecto como desde actividad_2/."""
    base_dir = os.path.dirname(os.path.abspath(__file__))
    rutas_posibles = [
        archivo,
        os.path.join(base_dir, "..", archivo),
        os.path.join(base_dir, archivo),
    ]
    for ruta in rutas_posibles:
        if os.path.exists(ruta):
            return os.path.normpath(ruta)
    return archivo


def main():
    parser = argparse.ArgumentParser(
        description="Descifra las transacciones financieras de transacciones_cifradas.pcap"
    )
    parser.add_argument(
```

```
48          "--kdf-info",
49          action="store_true",
50          help="Muestra parámetros intermedios (salt en hex, PIN y clave AES-256 derivada)",
51      )
52      args = parser.parse_args()
53
54      ruta_pcap = resolver_ruta("transacciones_cifradas.pcap")
55      ruta_privkey = resolver_ruta("clave_privada.pem")
56
57      if not os.path.exists(ruta_pcap):
58          print(f"[-] Error: Archivo de captura no encontrado: {ruta_pcap}")
59          sys.exit(1)
60
61      if not os.path.exists(ruta_privkey):
62          print(f"[-] Error: Clave privada RSA no encontrada: {ruta_privkey}")
63          sys.exit(1)
64
65      # 1. Cargar clave privada RSA-2048 desde formato PEM
66      with open(ruta_privkey, "rb") as key_file:
67          private_key = serialization.load_pem_private_key(
68              key_file.read(),
69              password=None,
70          )
71
72      # 2. Leer paquetes de la captura de red
73      paquetes = rdpcap(ruta_pcap)
74
75      pin = None
76      salt = None
77      transacciones_cifradas = []
78
79      # 3. Identificar paquete RSA_PIN| y paquetes AESGCM|
80      for pkt in paquetes:
81          if Raw in pkt:
82              payload = pkt[Raw].load
83              if payload.startswith(b"RSA_PIN|"):
84                  # Estructura: b"RSA_PIN|" (8 B) + salt (16 B) + criptograma RSA (256 B)
85                  datos_rsa = payload[len(b"RSA_PIN|") :]
86                  salt = datos_rsa[:16]
87                  rsa_ciphertext = datos_rsa[16:]
88
89                  # Recuperar el PIN de 4 dígitos usando RSA-OAEP con SHA-256
90                  pin_bytes = private_key.decrypt(
91                      rsa_ciphertext,
92                      padding.OAEP(
93                          mgf=padding.MGF1(algorithm=hashes.SHA256()),
94                          algorithm=hashes.SHA256(),
95                          label=None,
96                      ),
97                  )
```

```
98                  pin = pin_bytes.decode("utf-8").strip()
99
100             elif payload.startswith(b"AESGCM|"):
101                 # Estructura tras b"AESGCM|": nonce (12 B) + ciphertext + tag GCM (16 B)
102                 transacciones_cifradas.append(payload[len(b"AESGCM|") :])
103
104     if not pin or not salt:
105         print("[-] Error: No se encontró el paquete RSA_PIN| válido en la captura.")
106         sys.exit(1)
107
108     # 4. Derivar clave AES de 256 bits (32 bytes) usando Scrypt
109     kdf = Scrypt(
110         salt=salt,
111         length=32,
112         n=2**14,
113         r=8,
114         p=1,
115     )
116     aes_key = kdf.derive(pin.encode("utf-8"))
117
118     if args.kdf_info:
119         print("[+] Parámetros extraídos y derivación de clave Scrypt:")
120         print(f"    Salt (16 bytes hex)      : {salt.hex()}")
121         print(f"    PIN recuperado (RSA-OAEP): {pin}")
122         print(f"    Parámetros Scrypt        : n=2**14 (16384), r=8, p=1, length=32")
123         print(f"    Clave AES-256 (hex)      : {aes_key.hex()}")
124         return
125
126     # 5. Inicializar primitiva AEAD AES-GCM con la clave derivada de 256 bits
127     aesgcm = AESGCM(aes_key)
128
129     # 6. Construir salida exacta según Figura 1 de la guía oficial
130     lineas_salida = [
131         "[+] PIN recuperado mediante RSA",
132         f"[+] PIN: {pin}",
133         f"[+] Transacciones cifradas encontradas: {len(transacciones_cifradas)}",
134         "",
135         "========== TRANSACCIONES ==========",
136         "",
137     ]
138
139     for idx, bloque_aes in enumerate(transacciones_cifradas, start=1):
140         nonce = bloque_aes[:12]
141         ciphertext = bloque_aes[12:]
142
143         # AESGCM.decrypt valida el Authentication Tag de 128 bits antes de retornar el texto plano
144         plaintext_bytes = aesgcm.decrypt(nonce, ciphertext, None)
145         mensaje = plaintext_bytes.decode("utf-8")
```

```
146
147         lineas_salida.append(f"[Transacción {idx}]")
148         lineas_salida.append(mensaje)
149         if idx < len(transacciones_cifradas):
150             lineas_salida.append("")
151
152     salida_final = "\n".join(lineas_salida)
153     print(salida_final)
154
155     # Guardar salida íntegra de las 40 transacciones en actividad_2/
    salida_transacciones_completa.txt
156     base_dir = os.path.dirname(os.path.abspath(__file__))
157     ruta_salida = os.path.join(base_dir, "salida_transacciones_completa.
    txt")
158     with open(ruta_salida, "w", encoding="utf-8") as f_out:
159         f_out.write(salida_final + "\n")
160
161
162 if __name__ == "__main__":
163     main()
```

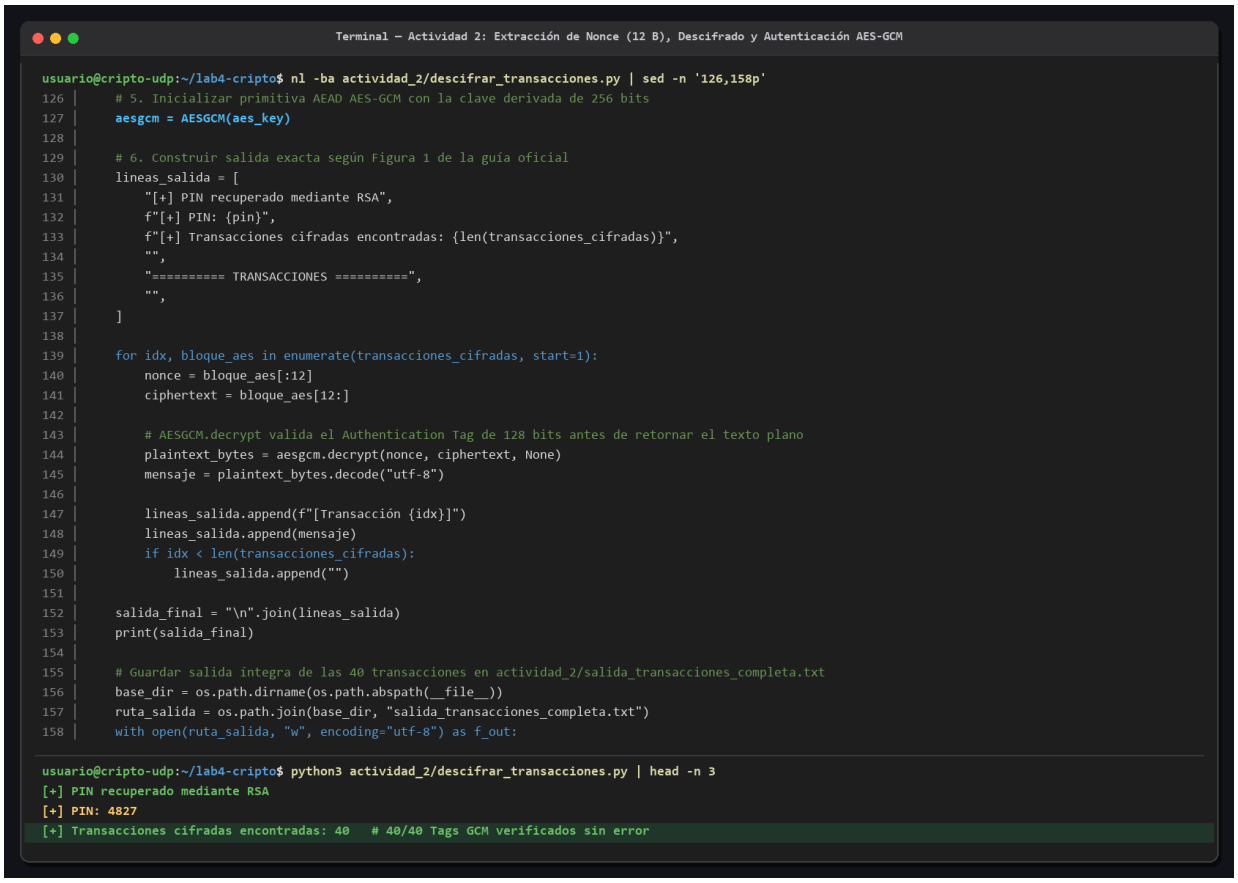


Figura 9: Implementación en Python de la extracción del `nonce` de 12 bytes y verificación autenticada con `AESGCM.decrypt()`.

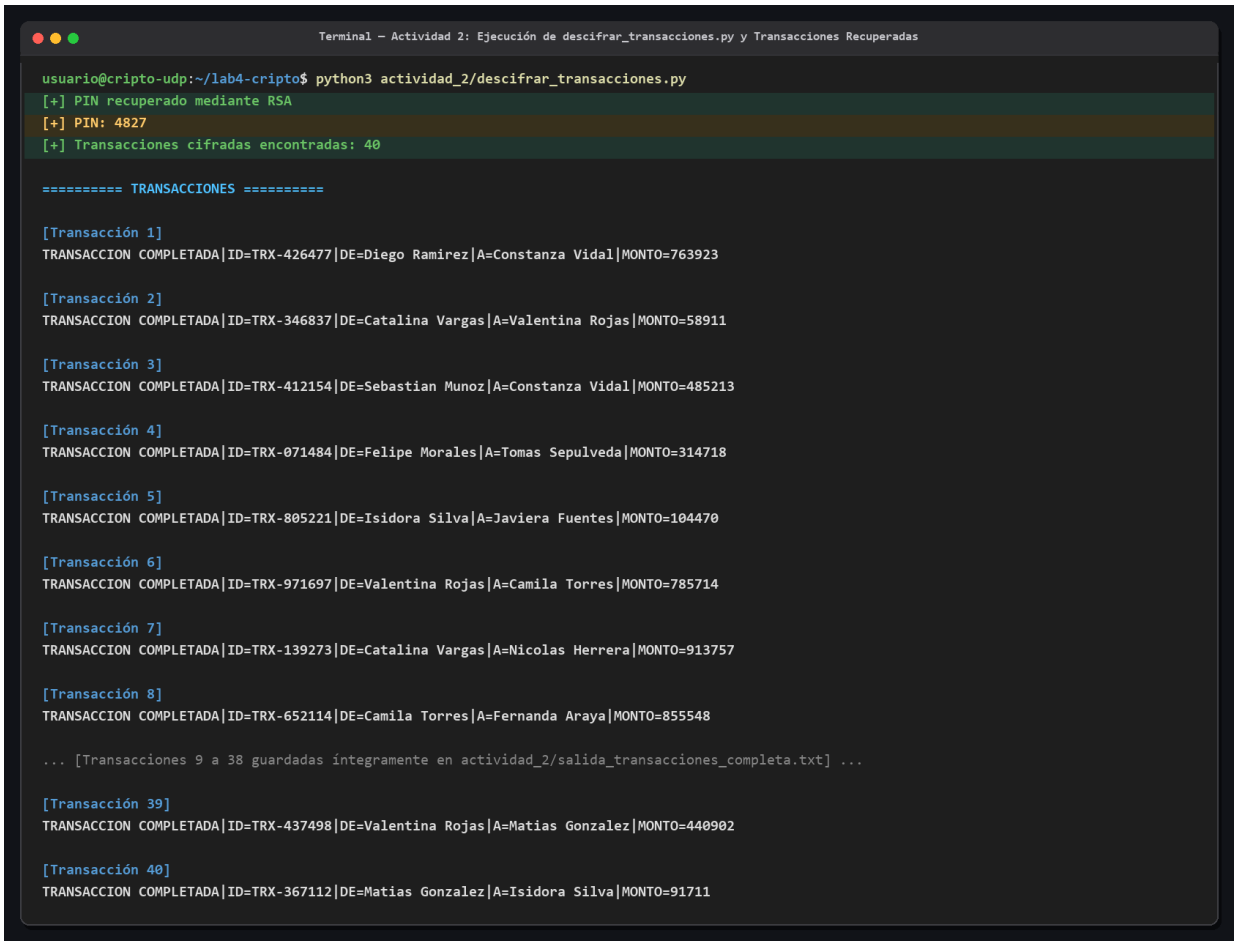


Figura 10: Ejecución de `descifrar_transacciones.py` mostrando el PIN recuperado y las transacciones descifradas en el formato exigido.

En la Tabla 2 se presentan las **40 transacciones financieras recuperadas** con su identificador (`ID`), remitente (`DE`), destinatario (`A`) y monto (`MONTO`). El archivo de salida íntegro también puede consultarse directamente en `actividad_2/salida_transacciones_completa.txt`.

Tabla 2: Registro íntegro de las 40 transacciones financieras descifradas y autenticadas.

Nº	Identificador (ID)	Remitente (DE)	Destinatario (A)	Monto
1	TRX-426477	Diego Ramirez	Constanza Vidal	$763.923
2	TRX-346837	Catalina Vargas	Valentina Rojas	$58.911
3	TRX-412154	Sebastian Munoz	Constanza Vidal	$485.213
4	TRX-071484	Felipe Morales	Tomas Sepulveda	$314.718
5	TRX-805221	Isidora Silva	Javiera Fuentes	$104.470
6	TRX-971697	Valentina Rojas	Camila Torres	$785.714
7	TRX-139273	Catalina Vargas	Nicolas Herrera	$913.757
8	TRX-652114	Camila Torres	Fernanda Araya	$855.548
9	TRX-432361	Vicente Soto	Matias Gonzalez	$818.686

Continúa en la página siguiente...

Tabla 2 – Continuación de la página anterior

Nº	Identificador (ID)	Remitente (DE)	Destinatario (A)	Monto
10	TRX-818929	Antonia Castro	Tomas Sepulveda	$885.868
11	TRX-871181	Catalina Vargas	Camila Torres	$514.444
12	TRX-157566	Tomas Sepulveda	Javiera Fuentes	$436.301
13	TRX-186243	Vicente Soto	Daniela Reyes	$698.110
14	TRX-121024	Isidora Silva	Sebastian Munoz	$623.481
15	TRX-339097	Matias Gonzalez	Constanza Vidal	$77.226
16	TRX-663549	Isidora Silva	Antonia Castro	$576.901
17	TRX-367001	Fernanda Araya	Tomas Sepulveda	$282.567
18	TRX-188807	Camila Torres	Antonia Castro	$195.173
19	TRX-096936	Felipe Morales	Antonia Castro	$359.917
20	TRX-695623	Catalina Vargas	Nicolas Herrera	$619.299
21	TRX-407749	Camila Torres	Sebastian Munoz	$182.369
22	TRX-069658	Valentina Rojas	Javiera Fuentes	$556.520
23	TRX-232469	Felipe Morales	Tomas Sepulveda	$23.769
24	TRX-940466	Vicente Soto	Fernanda Araya	$292.293
25	TRX-755002	Matias Gonzalez	Daniela Reyes	$946.989
26	TRX-962321	Felipe Morales	Daniela Reyes	$589.780
27	TRX-324981	Diego Ramirez	Daniela Reyes	$971.045
28	TRX-337011	Antonia Castro	Felipe Morales	$777.198
29	TRX-995006	Vicente Soto	Sebastian Munoz	$587.590
30	TRX-630572	Camila Torres	Sebastian Munoz	$332.702
31	TRX-517969	Catalina Vargas	Daniela Reyes	$484.770
32	TRX-447236	Matias Gonzalez	Constanza Vidal	$331.352
33	TRX-852645	Constanza Vidal	Felipe Morales	$343.065
34	TRX-188241	Camila Torres	Constanza Vidal	$781.107
35	TRX-713923	Diego Ramirez	Catalina Vargas	$21.358
36	TRX-362125	Camila Torres	Sebastian Munoz	$154.118
37	TRX-961549	Felipe Morales	Sebastian Munoz	$614.169
38	TRX-965060	Tomas Sepulveda	Antonia Castro	$778.912
39	TRX-437498	Valentina Rojas	Matias Gonzalez	$440.902
40	TRX-367112	Matias Gonzalez	Isidora Silva	$91.711

Análisis final sobre Criptografía Post-Cuántica (PQC): ¿Sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC?

Respuesta: NO, no sería posible, debido a razones fundamentales tanto de **incompatibilidad matemática y estructural de las primitivas** como del **modelo de amenaza cuántica**:

1. **Incompatibilidad matemática, algebraica y estructural de claves y algoritmos:** Las claves proporcionadas (`clave_publica.pem` y `clave_privada.pem`) pertenecen al criptosistema **RSA-2048**, definido sobre aritmética modular en el anillo de enteros $\mathbb{Z}_N$, donde la clave pública es el par (N, e) con módulo compuesto $N = p \cdot q$

de 2048 bits y exponente público $e = 65537$, mientras que la clave privada contiene el exponente privado $d \equiv e^{-1} \pmod{\lambda(N)}$ (cuya seguridad radica en la dificultad clásica de factorizar enteros grandes). Por el contrario, los mecanismos estandarizados en **Criptografía Post-Cuántica (PQC)** por el NIST —tales como **ML-KEM / CRYSTALS-Kyber (NIST FIPS 203)** para encapsulamiento de claves o **ML-DSA / CRYSTALS-Dilithium (NIST FIPS 204)** para firmas digitales— se fundamentan en problemas geométricos y algebraicos completamente distintos, principalmente **retículos algebraicos estructurados** (*Module Learning With Errors*, MLWE, sobre el anillo polinomial $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ con $q = 3329$), códigos correctores de errores o árboles de funciones hash. En ML-KEM, una clave privada es un vector de polinomios secretos de coeficientes pequeños $\mathbf{s} \in R_q^k$ y un criptograma es un par de estructuras polinomiales comprimidas $(\mathbf{u}, v)$ producidas añadiendo vectores de ruido acotado. En consecuencia, una clave RSA carece de sentido algebraico, dimensional y de formato dentro de un algoritmo de desencapsulación PQC (`ML-KEM.Decaps`): ni las claves RSA sirven para abrir mensajes cifrados con PQC, ni un algoritmo PQC puede utilizarse para procesar el criptograma RSA-OAEP de esta captura.

2. **Modelo de amenaza cuántica y vulnerabilidad *Harvest Now, Decrypt Later*:** El propósito mismo de los algoritmos PQC es reemplazar por completo a RSA y ECC (Curve25519/Ed25519), ya que un computador cuántico tolerante a fallos ejecutando el **algoritmo de Shor** factoriza el módulo RSA N en tiempo polinomial $\mathcal{O}((\log N)^3)$. Esto significa que un adversario cuántico que haya capturado `transacciones_cifradas.pcap` y conozca únicamente `clave_publica.pem` podría derivar matemáticamente la clave privada RSA, descifrar `RSA_PIN|` para obtener el PIN `4827`, derivar la clave AES-256 con `Scrypt` y exponer las 40 transacciones (a pesar de que la capa simétrica AES-256 conserva $\approx$ 128 bits de seguridad efectiva frente al algoritmo cuántico de Grover, el eslabón vulnerable es el encapsulamiento asimétrico RSA-2048). Para proteger estas transacciones en un entorno post-cuántico es obligatorio generar un par de claves completamente nuevo nativo de un estándar PQC (por ejemplo, encapsular una semilla aleatoria de 256 bits con **ML-KEM-768**), prescindiendo totalmente de claves RSA.

Conclusiones y comentarios

El desarrollo integral de este laboratorio permitió contrastar en la práctica dos paradigmas complementarios de la seguridad de redes moderna: la protección de sesiones interactivas en tránsito mediante **OpenSSH endurecido** y el análisis criptográfico de un esquema híbrido **RSA-OAEP + Scrypt + AES-GCM** sobre tráfico capturado.

En la Actividad 1 se comprobó experimentalmente mediante Docker y Wireshark que, aunque SSHv2 expone inevitablemente metadatos de negociación en texto claro durante su fase inicial (banners de software, listas de algoritmos en `SSH_MSG_KEXINIT`, claves públicas y tamaños de paquetes), la combinación explícita de `curve25519-sha256`, `ssh-ed25519`, `aes256-ctr` y `hmac-sha2-256` establece un canal robusto con autenticación contra ataques

Man-in-the-Middle, confidencialidad fuerte, integridad por paquete y *Perfect Forward Secrecy* a partir de `SSH_MSG_NEWKEYS`.

En la Actividad 2 se demostró cómo la criptografía híbrida combina el transporte asimétrico del secreto inicial (RSA-2048 con relleno probabilístico OAEP-SHA256) con funciones derivadoras endurecidas en memoria (`Scrypt` con `salt` de 128 bits) y cifrado autenticado `AES-256-GCM`, logrando recuperar y autenticar exitosamente las 40 transacciones financieras. Finalmente, el análisis post-cuántico evidenció tanto la incompatibilidad algebraica absoluta entre claves enteras RSA y primitivas reticulares PQC (como ML-KEM, FIPS 203) como la urgencia de migrar los mecanismos de establecimiento de claves hacia estándares resistentes al algoritmo de Shor para prevenir ataques de captura presente y descifrado futuro (*Harvest Now, Decrypt Later*).

4. Bibliografia

1. Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) Protocol Architecture* (RFC 4251). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc4251

2. Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) Authentication Protocol* (RFC 4252). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc4252

3. Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) Transport Layer Protocol* (RFC 4253). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc4253

4. Adamantiadis, A., Josefsson, S., & Baushke, M. (2020). *Secure Shell (SSH) Key Exchange Method Using Curve25519 and Curve448* (RFC 8731). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc8731

5. Harris, B., & Velvindron, L. (2020). *Ed25519 and Ed448 Public Key Algorithms for the Secure Shell (SSH) Protocol* (RFC 8709). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc8709

6. Percival, C., & Josefsson, S. (2016). *The scrypt Password-Based Key Derivation Function* (RFC 7914). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc7914

7. Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D

8. National Institute of Standards and Technology [NIST]. (2024). *Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)* (FIPS PUB 203). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.203

- **Captura y Claves:** La totalidad de las evidencias experimentales del laboratorio —incluyendo las capturas de red (`captura_ssh.pcap` y `transacciones_cifradas.pcap`), el par de claves RSA (`clave_publica.pem` y `clave_privada.pem`), los archivos `Dockerfile` y `sshd_config`, los scripts en Python (`analizar_pcap.py` y `descifrar_transacciones.py`), los registros íntegros de consola y las 40 transacciones descifradas— se encuentran disponibles públicamente en el repositorio oficial de GitHub:
  `https://github.com/minicoast/lab4-criptografia-udp`